20. Софтуерна архитектура. Проектиране и документиране на софтуерни архитектури.

1. *Понятие за софтуерна архитектура. Структури и изгледи (structures and views) на архитектурата. Необходимост от софтуерни архитектури. Влияние на архитектурата върху проекта и организацията.*

2. *Архитектурни стилове. Многослоен стил, Модел-изглед контролер (MVC), Поточни (Pipe-and-filter) архитектури. Архитектура ориентирана към услуги.*

3. *Проектиране на софтуерната архитектура. Процес за проектиране. Избор на подходящи структури. Последователност на създаване на архитектурата. Тактики (архитектурни решения) за постигане на желаните качествени показатели.*

4. *Документиране на софтуерната архитектура. Предназначение на документацията. Основен принцип на документиране, съдържание и структура на документацията*

1.1. Понятие за софтуерна архитектура

Съгласно Software Engineering Institute:

“Архитектура на дадена софтуерна система е съвкупност от *изгледи*, показващи различните софтуерни елементи на системата, външно видимите им свойства и връзките между тях”

The software architecture of a program or computing system is the structure or structures of the system, which comprise software elements, the externally visible properties of those elements, and the relationships among them.

1.1. Понятие за софтуерна архитектура

Софтуерната архитектура представлява:

- фундаменталната организация на системата
- съвкупност от архитектурни структури
- елементи, връзки и ограничения
- ключови проектни решения
- основа за развитие на системата

1.2. Защо е необходима архитектура?

Управление на сложността

- разделяне на системата на части
- по-лесно разбиране

Комуникация

- архитекти
- разработчици
- тестери
- клиенти

Основа за анализ

- качество
- риск
- цена
- срокове

Повторна употреба

- архитектурни шаблони
- доказани решения

1.3. Структури и изгледи

Structure

Реалната организация на системата.

View

Представяне на структурата за определена аудитория.

Една система има:

- много структури
- много изгледи

Нито един изглед не описва цялата архитектура.

1.4. Видове архитектурни структури

Архитектурните изгледи се делят най-общо казано на 3 групи:

- Модулни изгледи:
  - Декомпозиция на модулите
  - Употреба на модулите
  - Изглед на слоевете (многослоест)
  - Йерархия на класовете
- Изгледи на процесите;
- Изгледи на разположението;
  - Изглед на внедряването
  - Файлов изглед
  - Разпределение на работата

1.5. Моделът 4+1

1. **Логически изглед** - основните класове, компоненти и подсистеми.
2. **Изглед на процесите** - изпълняваните процеси и тяхното взаимодействие.
3. **Изглед на кода (Development View)** - организацията на програмния код.
4. **Физически изглед (Deployment View)** - разполагането върху хардуер.

(+1) Сценарии на употреба - примери за реална работа на системата.

1.6. Архитектура и организация

Architecture–Business Cycle

Бизнесът влияе върху архитектурата чрез:

- изисквания
- бюджет
- срокове

Архитектурата влияе върху:

- организацията
- риска
- разходите
- поддръжката

2. Архитектурни стилове. Многослоен стил, Модел-изглед контролер (MVC), Поточни (Pipe-and-filter) архитектури. Архитектура ориентирана към услуги

2.1. Архитектурни стилове

Набор от:

- компоненти
- конектори
- ограничения
- правила за взаимодействие

2.2. Многослоен стил (Layered Architecture)

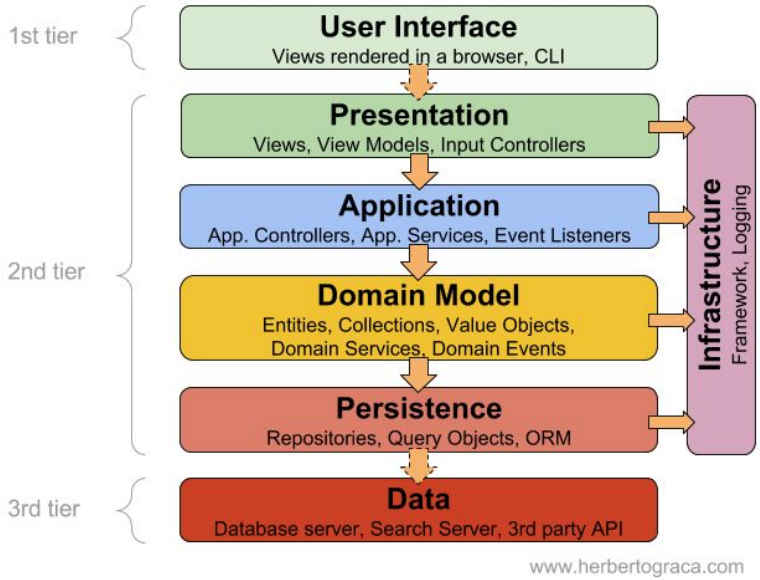
Слой - използва услуги само на слоя под него, предоставя услуги на слоя над него.

Предимства

- разделяне на отговорностите
- скриване на информацията
- висока кохезия
- лесна поддръжка
- лесно тестване

Недостатъци

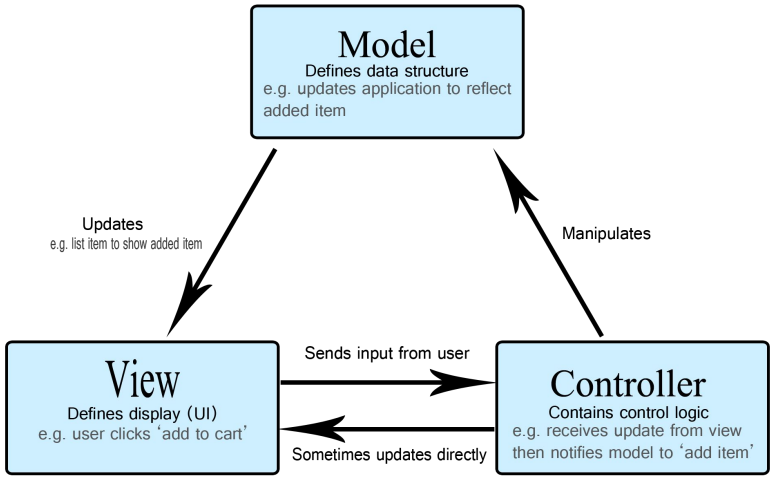
- допълнителни зависимости
- по-ниска производителност



2.3. Модел–изглед–контролер (MVC)

Разделя системата на 3 компонента:

- Model - данни и бизнес логика
- View - потребителски интерфейс
- Controller - обработва входа от потребителя



Предимства

- разделяне на отговорностите
- множество изгледи
- добра поддръжка

Недостатъци

- допълнителна сложност
- допълнителен код

2.4. Поточни архитектури (Pipe-and-Filter)

Системата се състои от последователни компоненти (filters), които:

- получават входни данни
- обработват ги
- подават резултата към следващия компонент

Свързването между тях се осъществява чрез **pipes**.

Input → Filter1 → Filter2 → Filter3 → … → Output

Предимства

- повторна употреба
- паралелизъм
- независими компоненти

Недостатъци

- слаба производителност
- трудно споделяне на глобални данни

2.5. Архитектура, ориентирана към услуги (Service-Oriented Architecture, SOA)

Основни участници

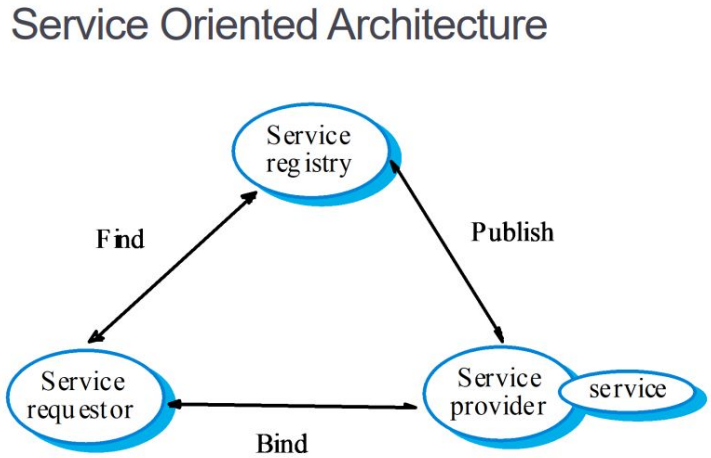
- Service Provider
- Service Registry
- Service Consumer

Характеристики

- слаба свързаност
- повторна употреба
- оперативна съвместимост

Основни операции

- Publish - Регистриране на услуга
- Find - Откриване на услуга
- Bind - Използване на услуга



3. Проектиране на софтуерната архитектура. Процес за проектиране. Избор на подходящи структури. Последователност на създаване на архитектурата. Тактики (архитектурни решения) за постигане на желаните качествени показатели.

3.1. Проектиране на архитектурата

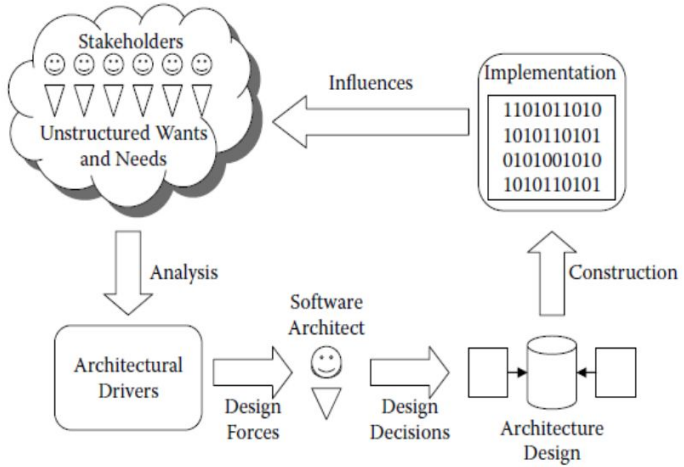
Входни данни

- функционални изисквания
- качествени изисквания
- ограничения

Основни стъпки

- Анализ на изискванията
- Избор на архитектурен стил
- Декомпозиция
- Интерфейси
- Оценка

Цикличен процес на създаване на архитектурата



3.2. Архитектурни драйвери

Функционални изисквания

- Какво трябва да прави системата?

Качествени показатели

- производителност
- сигурност
- надеждност

Бизнес ограничения

- срокове
- бюджет
- технологии

3.3. ADD (Attribute Driven Design)

Метод за проектиране на архитектурата на база качествени показатели.

Основни стъпки

1. Избор на елемент за декомпозиция
2. Избор на архитектурни драйвери
3. Избор на архитектурен стил
4. Създаване на подмодули
5. Дефиниране на интерфейси
6. Проверка на решенията

3.4. Същност на архитектурните тактики

- Архитектурно решение за постигане на конкретен качествен показател.
- Влияят директно върху качествените характеристики;
- Наборът от конкретни тактики се нарича **архитектурна стратегия**;

3.4.1. Тактики за производителност (performance)

- Намаляване на изискванията
  a. По-добри алгоритми
  b. Кеширане
  c. Намаляване на overhead
  d. Ограничаване на време

- Управление на ресурсите
  a. Паралелизъм
  b. Излишък (cache, replicas)
  c. Повече ресурси

- Арбитраж (Scheduling)
  a. FIFO
  b. Приоритети

3.4.2. Тактики за улесняване на тестването (Testability)

- Record & Replay
- Interface vs Implementation
- Test interface
- Мониторинг

3.4.3. Тактики за сигурност (Security)

- Устояване на атаки
  a. Authentication
  b. Authorization
  c. Encryption
  d. Integrity

- Възстановяване след атака
  a. Възстановяване на състояние
  b. Audit trail

3.4.4. Тактики за удобство при употребата (Usability)

- По време на изпълнение (за крайния потребител)
  a. Feedback (Обратна връзка)
  b. Undo / Cancel
  c. Multiple Views
  d. User Initiative vs System Initiative
- Свързани с UI (насочени към разработчика на интерфейса)

3.4.5. Тактики за изправност (Reliability)

- Откриване и предпазване от откази
  a. Ping / Echo
  b. Heartbeat
  c. Exceptions
- Отстраняване на откази
  a. Активен излишък (Active redundancy)
  b. Пасивен излишък (Passive redundancy)
  c. Извеждане от употреба
  d. Следене на процесите (Process Monitoring)
- Повторно въвеждане в употреба
  a. Паралелна работа (shadow mode)
  b. Контролни точки и rollback (Checkpoint/rollback)

3.4.6. Тактики за изменяемост (Modifiability)

- Локализиране на промените
  a. Поддръжка на семантична свързаност
  b. Очакване на промени
  c. Ограничаване на опции

- Предотвратяване на ефекта на вълната
  a. Скриване на информация
  b. Ограничена комуникация
  c. Посредници

- Отлагане на свързването
  a. Конфигурационни файлове
  b. Plug-and-Play Components

4. Документиране на софтуерната архитектура. Предназначение на документацията. Основен принцип на документиране, съдържание и структура на документацията

4.1. Документиране на архитектурата

Предназначение

- комуникация
- обучение
- анализ
- поддръжка
- съхраняване на знания

Основен принцип

Архитектурата се документира чрез множество изгледи.

4.2. Структура на архитектурната документация

Основна информация

- цели
- контекст
- терминология

Архитектурни изгледи

- модулни
- процесите
- разположението

Beyond Views

- архитектурни решения
- ограничения
- обосновка

4.3. Документиране на архитектурен изглед

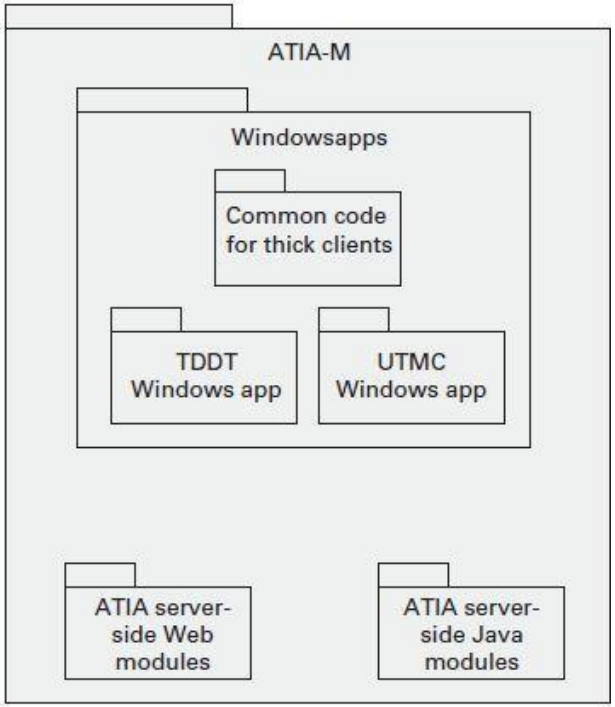
Всеки архитектурен изглед съдържа:

1. Основно представяне
2. Елементи и връзки
3. Контекст
4. Варианти на използване
5. Архитектурна обосновка
6. Терминологичен речник
7. Допълнителна информация

Припомняне на изгледи за задачи

Модулна декомпозиция

- Връзките между модулите са от вида **“X е под-модул на Y”**;
- Йерархично разделяне на системата на модули
- Всеки модул може да бъде допълнително декомпозиран на подмодули
- Всеки модул:
  - има ясно дефинирана отговорност
  - предоставя интерфейс
  - скрива вътрешната си реализация

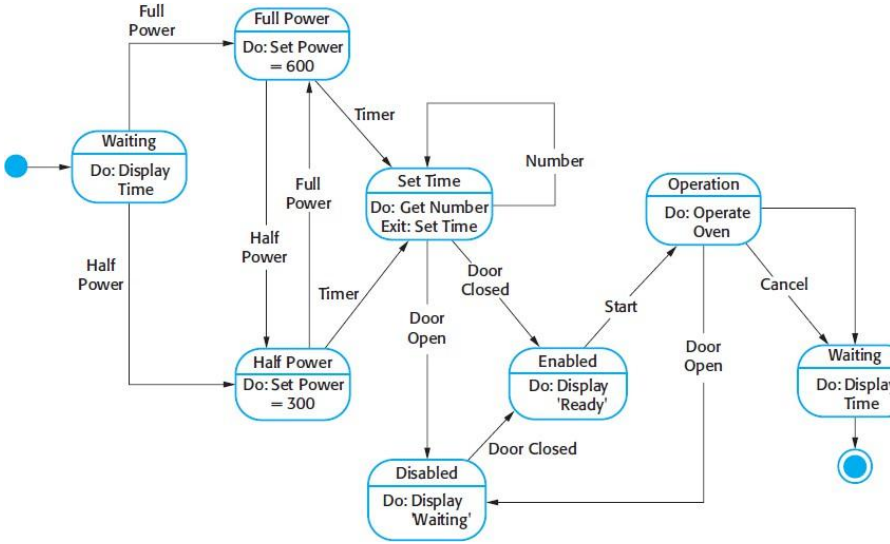


Изглед на процесите.

- Архитектурен изглед, при който елементите са процеси (или нишки), изпълнявани в системата (компоненти) и комуникационни, синхронизационни или блокиращи операции между тях (конектори);

- Връзките (attachments) показват:
  - как процесите комуникират
  - как са свързани с конекторите

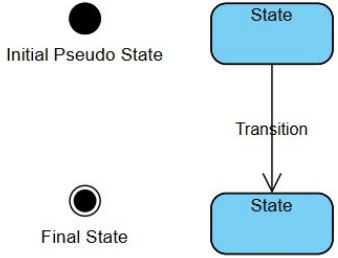
- Правим го след модулната декомпозиция, когато искаме да изобразим:
  - паралелно изпълнение
  - времеви ограничения
  - комуникация между компоненти



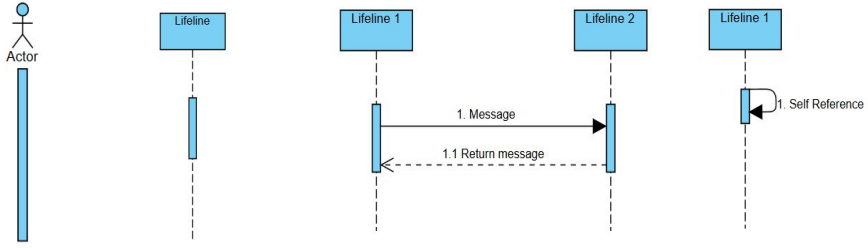
Как изобразяваме изгледа на процесите

- Най-често използваме следните UML диаграми да изобразим различните изгледи:
  - Sequence Diagram
  - Activity Diagram
  - State Machine Diagram

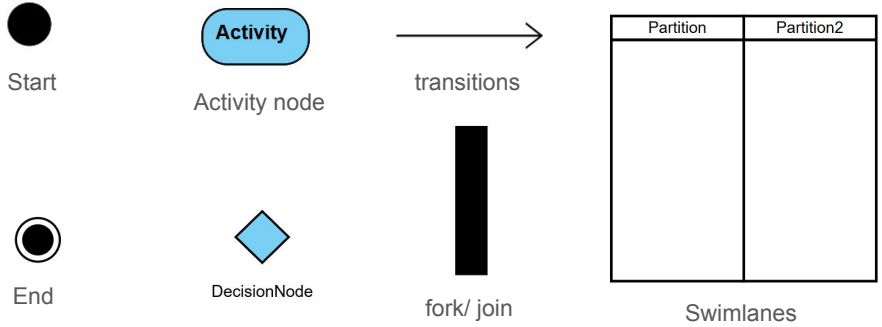
Simple State Machine Diagram Notation



Нотация при Sequence диаграмите



Activity Diagram



Избор на UML диаграма за представяне на изглед на процесите

Диаграма	Кога се използва	Какво показва	Типични случаи
Sequence Diagram	Когато има комуникация между процеси и ясен сценарий	Последователност от съобщения между процеси във времето	Клиент–сървър, API заявки, взаимодействие между компоненти
Activity Diagram	Когато има поток от действия, разклонения	Последователност от действия и паралелни потоци	Бизнес процеси, обработка на заявки, pipeline
State Machine Diagram	Когато системата има състояния и реагира на събития	Състояния и преходи между тях	Устройства, контролни системи, протоколи

Изглед на разположението

Архитектурен изглед, който показва как софтуерните елементи се разполагат върху физическата среда, в която системата работи.

- показва връзката между софтуера и хардуера;
- описва къде се изпълняват различните компоненти;
- представя комуникацията между устройства и среди;

Правим го след като вече имаме:

- модулна декомпозиция;
- изглед на процесите;
- ясно дефинирани компоненти.

